Table of Contents

Stock Seasonality Analyzer: A Reproducible, Survivorship-Aware Framework for Calendar and Event-Window Analysis of Consumer-Spending Equities

Author: Prince Verma **Date:** May 2026 **Status:** Working paper / system description

---

Abstract

Calendar effects and event-window anomalies are well-documented features of equity returns (Rozeff & Kinney, 1976; Brown & Warner, 1985; Bouman & Jacobsen, 2002), but accessible, transparent tooling for retail and educational analysis remains limited — practitioners typically choose between expensive professional terminals and opaque commercial dashboards. We present **Stock Seasonality Analyzer**, an open-source web application that computes and visualizes monthly seasonality and festival-window returns for a curated universe of consumer-spending equities over a 15-year trailing window. The system ingests ~303,000 daily OHLCV bars across 89 active and 6 delisted tickers, computes ~15,000 cached seasonality and event-window statistics, and serves them through a server-rendered web interface where every percentage is reported alongside its underlying sample size. Methodologically, we (1) implement survivorship-aware ingestion that preserves historical data for delisted tickers, (2) deliberately use deterministic descriptive statistics rather than machine learning given small sample sizes (5–15 years), and (3) cache all computed scores so the analysis is never recomputed on a request path. The contribution

is a methodology and reference implementation that prioritizes interpretability and statistical honesty over predictive complexity. All code, schema, and data pipeline are reproducible from a public Git repository.

Keywords: stock seasonality, event-window analysis, survivorship bias, descriptive statistics, financial software engineering, consumer-spending equities, reproducible research

---

1. Introduction

The observation that equity returns vary systematically with the calendar — the "January effect" (Rozeff & Kinney, 1976), the "Halloween indicator" (Bouman & Jacobsen, 2002), turn-of-the-month effects, holiday effects, and others — has been one of the most-studied anomalies in empirical finance. Practitioner tooling for analyzing these effects, however, is concentrated in a small number of expensive commercial products: Bloomberg Terminal, FactSet, and dedicated seasonality dashboards such as Seasonax and Equity Clock. These tools are powerful but unsuitable for educational use, independent research, or methodological scrutiny: their data sources are opaque, their computation methods rarely documented, and their interfaces designed for professional traders rather than students or curious retail participants.

We were motivated by two observations:

1. **The math is simple, but the engineering is non-trivial.** Computing average monthly returns is a one-line operation; doing so correctly across 15 years of trading data, with survivorship bias controlled, ticker churn handled, sample sizes surfaced, and results cached for sub-second page loads, is a several-week engineering project.
2. **The "right" results for descriptive seasonality are deterministic.** Given the same input data and the same window, two correct implementations must agree to the last decimal place. This makes the project a good fit for an open, reproducible reference implementation — one whose computations can be independently audited.

We therefore built a complete system: data ingestion, analysis engine, REST API, and web user interface. The contribution of this paper is not a new statistical method or a new empirical finding — both calendar effects and event-window methodology are decades-old in the literature — but a methodological synthesis and a working, fully reproducible reference implementation that emphasizes:

- **Statistical honesty over predictive complexity.** Every reported percentage is accompanied by its sample size; no machine-learning prediction or smoothing is applied.
- **Survivorship awareness as a first-class schema concern.** Delisted tickers remain in the historical record with a flag, not silently dropped from the active universe.
- **Aggressive caching of deterministic computations.** Scores are computed once after each data refresh and read from a database on every subsequent request.

The remainder of this paper is organized as follows. Section 2 reviews related literature. Section 3 describes the data: universe selection, sources, and quality controls. Section 4 specifies the analytical methods. Section 5 presents the system architecture. Section 6 reports implementation details and selected outputs. Section 7 discusses limitations. Section 8 outlines future work.

---

2. Related Work

2.1 Calendar effects in equity returns

The literature on calendar anomalies is large and well-established. Rozeff and Kinney (1976) first documented the January effect — disproportionate small-cap returns in the first calendar month of the year — in monthly NYSE returns from 1904 to 1974. Keim (1983) linked the January effect to firm size. Haugen and Lakonishok (1988) extended the analysis through the 1980s. Lakonishok and Smidt (1988) examined a century of Dow Jones daily data and documented turn-of-the-month, turn-of-the-year, and holiday effects.

The "Halloween indicator" or "Sell in May and go away" pattern — equity returns being substantially lower from May through October than from November through April — was formalized by Bouman and Jacobsen (2002), who documented the effect in 36 of 37 country indices studied. Subsequent work (Andrade, Chhaochharia, & Fuerst, 2013) extended and stress-tested the finding across additional markets and decades.

Cross-sectional seasonality (the observation that within-month return rankings persist across years) was documented by Heston and Sadka (2008). Cao and Wei (2005) connected seasonal patterns to weather and mood. Empirical evidence on holiday-specific effects in retail-relevant equities is sparser in the academic literature, though Ariel (1987) documented monthly patterns and Kim and Park (1994) examined holidays specifically.

The present work does not produce new findings within this literature. It implements a measurement framework consistent with this body of work, in particular the convention of reporting (a) the average return for each calendar month, (b) the fraction of years that month was positive, and (c) the sample size — all standard in the seasonality literature.

2.2 Event-window methodology

Brown and Warner (1985) and MacKinlay (1997) formalized the event-study methodology that became standard in empirical finance: define an event date T, define windows relative to T (e.g., $[T-30, T-1]$, $[T, T+1]$, $[T+1, T+10]$), and compute average returns or abnormal returns over those windows across multiple event occurrences. Although event studies originated in corporate-action research (earnings announcements, mergers, splits), the same framework applies cleanly to recurring calendar events such as Diwali, Christmas, or Black Friday.

Our implementation uses **trading-day** offsets (rather than calendar-day offsets) within event windows, consistent with the practice in Brown and Warner (1985), as this avoids the weekend/holiday artifact in calendar-day windows.

2.3 Survivorship bias

Brown, Goetzmann, Ibbotson, and Ross (1992) demonstrated that backtests run on currently-traded equities — silently excluding firms that delisted during the sample period — systematically overstate historical returns. Elton, Gruber, and Blake (1996) made the analogous point for mutual fund studies. The standard correction is to maintain a "point-in-time" universe in which a stock present from year t_1 to t_2 is included in analyses spanning those years and excluded thereafter.

Our schema encodes this directly: every `Stock` row carries `delisted` (boolean) and `delistedAt` (date) fields. The analysis pipeline does not drop delisted tickers from historical computations but flags them in the UI so users can interpret the result accordingly.

2.4 Software systems for financial analysis

System papers describing the architecture of financial analysis tools are less common in the academic literature than empirical studies, with notable exceptions including Bollerslev et al. (2009) on realized-volatility infrastructure and various technical reports from quantitative finance groups. Open-source platforms such as QuantConnect, Backtrader, and Zipline focus on backtesting and execution rather than descriptive seasonality. We are not aware of an open-source counterpart to our system that focuses specifically on calendar and event-window descriptive analysis with explicit survivorship handling and sample-size reporting.

---

3. Data

3.1 Universe

We selected 95 tickers across 9 consumer-spending categories, of which 89 are currently active and 6 are flagged delisted. Categories and their populations:

Category	Active	Delisted	Total
Travel	12	0	12
Retail	14	1 (JWN)	15
Food & beverage	12	0	12
Restaurants	10	0	10
Apparel	12	1 (GPS)	13
Jewelry & luxury	6	1 (TIF)	7
Cinema & entertainment	11	3 (PARA, SIX, SEAS)	14
Toys & hobby	5	0	5
Beauty & personal care	8	0	8
Total	**89**	**6**	**95**

The universe was constructed by category-relevant U.S. equity selection rather than market-cap screen; the goal was thematic coverage of consumer spending categories likely to exhibit calendar-driven demand patterns, not a representative slice of the broader market.

3.2 Sources

Price data: Daily OHLCV bars from Yahoo Finance via the open-source `yfinance` Python library (version 0.2.66). For each ticker we requested the maximum-available history capped at 15 years from the ingestion date, yielding approximately 252 trading days per year per ticker.

Festival data: Twenty-two-year coverage (2010–2031) of seventeen events seeded directly into the database:

- Fixed-date holidays: Christmas, Halloween, Valentine's Day, Independence Day.
- Computed nth-weekday holidays: Thanksgiving (4th Thursday of November), Black Friday and Cyber Monday (derived from Thanksgiving), Mother's Day (2nd Sunday of May), Father's Day (3rd Sunday of June), Memorial Day (last Monday of May), Labor Day (1st Monday of September).

- Lunar / movable feasts: Diwali (Drik Panchang), Chinese New Year (timeanddate.com), Easter (Western/Gregorian), Super Bowl Sunday (NFL historical schedule), all encoded as 22-row lookup tables.
- Seasonal anchors: back-to-school (Aug 15 anchor, 30-day duration).
- Asian e-commerce: Singles' Day (Nov 11).

Each festival is associated with a subset of categories via a many-to-many relation carrying a weight in [0, 1] expressing the strength of the historical effect (e.g., Diwali → Jewelry & Luxury = 1.0, Diwali → Apparel = 0.6).

3.3 Data quality controls

Adjusted close. All return calculations use Yahoo Finance's split- and dividend-adjusted close, which back-applies corporate actions to maintain time-series continuity. Raw OHLC values are retained in the database for charting and audit but are never used for return computation.

NaN filtering. The ingestion script drops any bar where `open`, `high`, `low`, `close`, or `adjusted close` is null (an occasional yfinance artifact for partial trading days or early IPO dates).

Ticker churn. Over the project's ingestion period (2026) we observed five tickers that ceased to return historical data under their original symbol due to corporate actions:

Old ticker	Disposition	Replacement	Date
TIF	Acquired (LVMH)	— (no replacement; data unavailable post-acquisition)	2021-01-07
GPS	Renamed (Gap Inc.)	GAP	2024-02-28
SEAS	Renamed (SeaWorld → United Parks)	PRKS	2024-02-08
SIX	Merged (Six Flags + Cedar Fair)	FUN	2024-07-01
JWN	Taken private (Nordstrom family + Liverpool)	—	2025-05-20
PARA	Merged (Paramount + Skydance)	PSKY	2025-08-07

In each case the legacy ticker was retained in the universe with `delisted = true` and `delistedAt` set to the dispositive date. Where the surviving entity retained historical data under a new ticker (PRKS, FUN, GAP, PSKY), the new ticker was added to the active universe. This treatment preserves the survivorship audit trail while ensuring the active universe reflects current market reality.

Source provenance. Every `PriceHistory` row carries a `source` column (default `"yfinance"`). Future ingestion from alternative providers (e.g., Polygon.io) would populate this column differently, allowing analyses to filter or compare by data source.

3.4 Resulting dataset

After ingestion, the database contains:

- **303,841 daily OHLCV rows** across 87 tickers that returned data (89 active + 6 delisted − the 4 tickers with no yfinance history: PARA, SEAS, SIX as old symbols and TIF as delisted with no history available; the new symbols PRKS / FUN / GAP / PSKY all returned full 15-year backfilled history).
- **6,336 cached seasonality scores** = 12 months × 3 window lengths (5y, 10y, 15y) × 2 COVID-toggle values × 88 stocks with data − rows with zero sample size.
- **8,976 cached event-window scores** = 17 festival slugs × 3 window kinds × 2 COVID-toggle values × 88 stocks − rows with zero sample size.
- **374 festival occurrences** across 22 years (2010–2031) covering the 15-year lookback plus 5-year forward calendar.

---

4. Methods

4.1 Monthly seasonality

Let `bars(s)` denote the time-ordered sequence of `PriceBar` records for stock s, where each `PriceBar` has fields `date` and `adjClose`. Given a trailing window of W years ($\in \{5, 10, 15\}$) measured backward from a reference date t_0 (the "as-of" date, defaulting to ingestion time), and an optional COVID-exclusion flag c, define:

$$\text{barsIn}(s, W, c, t_0) = \{b \in \text{bars}(s) : t_0 - W \text{ years} \le b.\text{date} \le t_0, \text{ and } b.\text{date} \notin [2020,2021] \text{ if } c = \text{true}\}$$

Partition `barsIn(s, W, c, t₀)` by calendar month (YYYY-MM), and define the monthly return for each non-singleton partition as

$$r(s, y, m) = \frac{\text{last}(\text{bars}_{y,m}).\text{adjClose} - \text{first}(\text{bars}_{y,m}).\text{adjClose}}{\text{first}(\text{bars}_{y,m}).\text{adjClose}}$$

The seasonality score for (stock s, month m, window W, COVID-flag c) is then the tuple:

- **avgReturn** = mean of `r(s, y, m)` over all years y in the window with valid data.
- **pctYearsPositive** = fraction of years y with `r(s, y, m) > 0`.
- **pctYearsBeatMean** = fraction of years y with `r(s, y, m) > μ(s, W, c)`, where μ is the mean of all monthly returns for s in the window.
- **sampleSize** = count of years y contributing a valid return.

The metric `pctYearsBeatMean` deserves comment. The seasonality literature sometimes asks "what fraction of years did this month outperform the average month for this stock?" Comparing each January return to the average of all Januaries would yield ~50% trivially, so the more meaningful comparison — the one that matches the literature's intent — is against the stock's overall mean monthly return within the window. Months with a structurally positive seasonal effect should beat that baseline more than half the time.

4.2 Event-window returns

For an event with occurrences at dates $E = \{e_1, e_2, \ldots, e_n\}$, and a stock s with sorted bars `bars(s)`, define the trading-day anchor index for event e_i as the smallest index k such that `bars(s)[k].date ≥ eᵢ`. (When the event falls on a non-trading day such as Christmas falling on a Sunday, the anchor is the first subsequent trading day, consistent with the convention in Brown and Warner, 1985.)

For each window kind ∈ {PRE30_PRE7, PRE7_EVENT, EVENT_POST7} we define index offsets:

- **PRE30_PRE7**: start offset −30 trading days, end offset −7 trading days from the anchor.
- **PRE7_EVENT**: start offset −7 trading days, end offset 0 (the event day itself) from the anchor.
- **EVENT_POST7**: start offset 0, end offset +7 trading days from the anchor.

The event-window return is computed as the percent return between the bars at the start and end offsets. Returns are aggregated across all occurrences within the window:

- **avgReturn** = mean of event-window returns across *E*.
- **pctYearsPositive** = fraction of occurrences with positive return.
- **sampleSize** = count of occurrences with sufficient data on both anchors (windows extending past the available data are dropped).

4.3 Survivorship handling

We do not implement abnormal-return adjustment (subtraction of a market benchmark or factor-model expected return) at the present stage. All reported returns are raw total returns. Delisted stocks contribute to historical analyses only for the period in which they had data; specifically, a delisted stock with `delistedAt = D` will contribute zero observations to windows beginning after *D* (the `sampleSize` for those scores will reflect the absence). The cached score tables only include rows where `sampleSize > 0`, so a delisted stock does not appear in analyses that would have required data beyond its delisting date.

4.4 Choice of descriptive statistics over machine learning

The system deliberately uses descriptive statistics — arithmetic means, fractions, and counts — rather than fitted machine-learning models. Three considerations motivated this choice:

1. **Sample sizes are inherently small.** With 5–15 years per (stock, month, window) cell, classical machine-learning estimators would overfit. The variance reduction from ML methods at this scale would be illusory — most of the bias-variance tradeoff has been resolved by the choice of estimator (the sample mean) rather than by tuning.
2. **The audience and use case favor interpretability.** The system is designed for educational and research use, where users benefit from understanding exactly what each number represents. A reported "+1.86%, 67% positive years, n = 15" is directly auditable; a model prediction of "+1.2% ± 0.4% with 70% confidence" hides where the number came from.
3. **The system explicitly disclaims predictive intent.** The disclaimer present on every page reads "Not investment advice. Past patterns do not guarantee future results." Building a predictive ML model would implicitly contradict this statement.

What we do *not* do — but could, in principle — is reported in Section 8.

---

5. System Architecture

The system is a Next.js 14 monolith backed by PostgreSQL, with a satellite Python ingestion process scheduled via GitHub Actions. The defining architectural principle is the **brief's "cache aggressively" rule**: raw prices are pulled once a day; analysis scores are computed offline and stored; pages read pre-computed scores with no math on the request path.

5.1 Topology

The system is organized into the following layers:

1. **Python ingestion** (`ingest_prices.py`). Pulls daily OHLCV via yfinance, upserts to `PriceHistory` via psycopg with `ON CONFLICT DO UPDATE`. Per-ticker transactions ensure that one bad symbol does not lose progress for others. Scheduled via a GitHub Actions cron job (Tue–Sat at 09:30 UTC, 30 minutes after the previous NYSE close).
2. **PostgreSQL on Supabase** (eu-west-1 session-mode pooler, port 5432). Eight tables: `Category`, `Stock`, `PriceHistory`, `FestivalEvent`, `FestivalEventCategory`, `SeasonalityScore`, `EventWindowScore`, `WaitlistSignup`. Schema managed by Prisma migrations.
3. **TypeScript analysis engine** (`src/lib/analysis/`). Pure functions for returns, monthly seasonality, and event-window aggregation. No I/O dependency; fully unit-tested (50 tests). Orchestrator (`run.ts`) reads `PriceHistory`, computes scores, atomically replaces the cached scores in `SeasonalityScore` and `EventWindowScore` via `prisma.$transaction([deleteMany, createMany, deleteMany, createMany])`. Invoked manually or via `npm run analyze`.
4. **TypeScript data layer** (`src/lib/data/`). Thin Prisma wrappers exposing `getMonthlyScores`, `getEventScores`, `getCategoryMonthlyAggregate`, etc. Used by both API routes and server components.
5. **Next.js API routes** (`src/app/api/`). Five endpoints: `GET /api/categories`, `GET /api/category/[slug]`, `GET /api/stock/[ticker]`, `GET /api/events/upcoming`, `POST /api/waitlist`. All read endpoints carry `Cache-Control: s-maxage=3600, stale-while-revalidate=86400` headers for CDN edge caching.
6. **Next.js server components** (`src/app/*/page.tsx`). Render the user-facing pages by calling the data layer directly. No internal HTTP fetch.
7. **Browser-side components.** Mostly server-rendered HTML; the single client-side interactive component is the analysis-controls toolbar, which writes window and COVID-toggle state to URL search parameters.

5.2 Database schema

The schema comprises eight models. The two cached-score tables (`SeasonalityScore` and `EventWindowScore`) have composite uniqueness constraints corresponding to the (stock, month, window, COVID-flag) and (stock, festival-slug, window-kind, COVID-flag) tuples respectively. The `Stock` model carries `delisted` and `delistedAt` for survivorship handling and `dataSource` and `sourceMeta` for provenance.

The festival schema decouples festival identity (e.g., “diwali”) from occurrence (one row per year). A many-to-many through-table `FestivalEventCategory` carries per-festival category-effect weights in [0, 1].

5.3 Caching strategy

There are three caching layers:

1. **Database-level cache** of computed scores. The `SeasonalityScore` and `EventWindowScore` tables are written by the analysis engine and read by both API routes and server components. They are the source of truth for all displayed percentages and never recomputed on the request path.

2. **HTTP cache headers** on API routes. `Cache-Control: public, s-maxage=3600, stale-while-revalidate=86400` instructs the CDN to serve cached responses for one hour and stale-while-revalidate responses for an additional 23 hours. Combined with the cache-key being the (URL, query-params) tuple, this means a popular page (`/stock/NKE?window=15`) is served from the edge most of the time.
3. **Browser cache.** Standard browser caching applies to static assets (CSS, fonts, JavaScript chunks) but not to the dynamic page HTML.

5.4 Survivorship at the schema level

The combination of `delisted` and `delistedAt` on `Stock` plus `sampleSize` on the score tables produces correct survivorship-aware analyses by construction. The analysis engine considers all stocks (including delisted) but emits score rows only when the relevant time window had data. A delisted stock that delisted before the analysis window has all-zero `sampleSize` cells, which are dropped by the engine and absent from the displayed results — without the analyst needing to remember to exclude them.

---

6. Implementation and Results

6.1 Technology stack

Layer	Technology	Justification
Web framework	Next.js 14 (App Router)	Server components allow direct database access without HTTP round-trips; static + dynamic rendering co-exist; mature ecosystem.
Language (web)	TypeScript 5	Type safety across the schema/data/API/UI boundary.
Language (ingestion)	Python 3.14	The yfinance and pandas ecosystem is Python-only.
Database	PostgreSQL on Supabase	Relational fit for the schema; reliable hosted Postgres; managed pooling.
ORM	Prisma 7 (driver-adapter generator)	Schema-as-code, type-generation, migrations; the new driver-adapter generator gives us the pg client transport without giving up type safety.
Charts (sparklines)	Hand-rolled SVG polylines	12-point sparklines do not justify a charting library.
Charts (heatmaps)	Custom CSS Grid with computed RGB cells	No public heatmap component in mainstream React chart libraries matches the diverging-palette and accessibility requirements (`role="cell"` per cell, native title tooltips, no client JS).
Tests	Vitest	Modern Jest-replacement with

Layer	Technology	Justification
		first-class TypeScript and ESM support; ~700 ms cold run for 66 tests.
Scheduling	GitHub Actions cron	Lightweight, no separate worker infrastructure; the daily refresh is a 90-second job.

6.2 Test coverage

The pure-function analysis library (`src/lib/analysis/returns.ts`, `seasonality.ts`, `event-window.ts`) carries 50 unit tests, including:

- Basic correctness on synthetic time series with known properties.
- Edge cases: empty input, single-bar months, zero adjusted-close prices, infinite/NaN inputs, dates outside the window.
- COVID-exclusion correctness (excluding 2020 and 2021 must reduce sample sizes deterministically).
- Trading-day binary search at exact, before-data, after-data, and gap-falling targets.
- Unsorted input is handled defensively by internal sorting.

The rate-limiting helper carries an additional 16 tests covering email validation, IP extraction from `X-Forwarded-For` and `X-Real-IP` headers, HMAC-SHA256 hashing properties, and the allow/deny decision rule. Total: 66 unit tests.

6.3 Performance

End-to-end timings observed on the production database (Supabase eu-west-1) from a residential connection:

Operation	Time	Notes
Daily price refresh (5-day window, 89 tickers)	6–8 seconds	Sequential ingestion
Full 15-year backfill (89 tickers, ~3700 bars each)	~85 seconds	Sequential; yfinance is the bottleneck
Full analysis run (89 active tickers, all windows, both COVID flags)	~30 seconds	Per-stock transaction; ~4 SQL round-trips per stock
Cold page render of `/stock/NKE`	200–350 ms	Dominated by DB query latency
Cold page render of `/category/jewelry-luxury`	250–400 ms	14 DB queries in parallel via `Promise.all`
API GET `/api/stock/NKE` (server-side)	100–200 ms	Excludes network round-trip
Unit test suite (cold)	1.3–2.0 seconds	66 tests across 4 files
Production build (`npm run build`)	30–45 seconds	Clean build of 14 routes

6.4 Example output: Nike (NKE)

To illustrate the output the system produces, Table 2 shows selected monthly seasonality scores for Nike (NKE) over the trailing 15-year window with COVID years included:

Table 2. Selected NKE monthly seasonality, 15-year window, COVID included.

Month	avgReturn	pctYearsPositive	pctYearsBeatMean	n
January	+0.43%	53%	47%	15
March	+1.21%	60%	53%	15
July	+1.86%	67%	67%	15
October	−0.47%	47%	40%	15
December	+1.04%	60%	53%	15

The headline observation is that July is the most seasonally favorable month for NKE in the sample (+1.86% average return, positive in 10 of 15 years, beating the stock's overall mean monthly return in 10 of 15 years), while October is the weakest. These directions are consistent with the well-known summer-strength / autumn-weakness pattern in consumer-discretionary equities (Heston & Sadka, 2008) and with NKE's heavy back-to-school exposure.

The system also computes event-window returns for the same stock against the back-to-school festival (anchored at August 15 of each year with a 30-day duration). The PRE30_PRE7 window (∼30 to ∼7 trading days before August 15, i.e., the early-July to early-August run-up) shows an average return of +1.31% across 15 occurrences, positive in 8 of 15.

We emphasize that these numbers are **historical descriptions**, not predictions. The 67% positive figure for NKE July does not imply a 67% probability that next July will be positive.

6.5 Example output: Travel category around Chinese New Year

The system also aggregates per-category statistics. For the Travel category (12 active stocks, primarily airlines, hotels, and cruise lines), the run-up to Chinese New Year (PRE30_PRE7 window across 22 historical occurrences) shows a category-wide average return of approximately +3.2% with an average sample size of 11 stocks per occurrence (varying by stock IPO date). The headline travel beneficiaries are Royal Caribbean (RCL) and Marriott (MAR), consistent with the established literature on the demand-side effect of the Chinese Lunar New Year on global tourism. Conversely, RCL also shows the highest within-Travel-category January seasonality (+13.3% over the 5-year trailing window, positive in 4 of 5 years).

We reiterate that these are descriptive observations on the present sample, not findings new to the literature, and not investment advice.

---

7. Discussion and Limitations

7.1 Sample sizes are small

The longest available window in the system is 15 years, giving at most 15 observations per (stock, month) cell. Many cells have fewer than 15 (newer IPOs, delisted stocks). At this scale, the variance of the sample mean is high and conventional statistical inference (p-values, confidence intervals) must be

interpreted with corresponding caution. We surface `sampleSize` on every percentage to make this visible. We deliberately do not display p-values, on the grounds that conventional p-values are misleading at $n \leq 15$ and may convey false precision.

7.2 No abnormal-return adjustment

All reported returns are raw — not market-relative, not factor-adjusted, not volatility-scaled. A positive January return for NKE may reflect a Nike-specific seasonal pattern, a broader market drift, or both. Disentangling the two would require subtracting an SPY (or factor-model) return computed over the same window — a meaningful addition we have flagged for future work but did not implement in the present version.

7.3 ADR currency mixing

Six of the 89 active tickers are American Depositary Receipts (ADRs): LVMUY (LVMH), CFRUY (Richemont), BURBY (Burberry), PRDSY (Prada), ADDYY (Adidas), LRLCY (L'Oréal). These trade in USD but track underlying prices denominated in EUR or GBP. Event-window returns for these tickers therefore mix the underlying stock's move with the FX move over the same window. For multi-week windows around major events this can be material. The `/about` page surfaces this caveat to users; the analysis does not correct for it.

7.4 yfinance data reliability

yfinance is the right choice for an open prototype but has documented limitations: occasional partial-bar artifacts, gaps for recently-renamed or single-letter tickers (we currently observe an empty response for "K" = Kellanova despite its active status), and column-shape changes across versions. Our schema (`dataSource`, `sourceMeta`) supports swapping to a paid provider (Polygon.io, EOD Historical Data) without a rewrite.

7.5 No backtest engine

A backtest would translate a seasonality observation ("Nike Jul is strong") into a simulated trading rule ("buy NKE on June 30, sell on July 31, every year") and compute the resulting portfolio return. We have placeholder UI on the stock-detail page but have not implemented the underlying logic, which requires position-sizing assumptions and an IRR calculation against historical `PriceHistory`. This is a substantial follow-up, primarily because it makes design choices (entry/exit rules, transaction-cost model) that have first-order effects on the result.

7.6 Universe bias

The 89-ticker universe is deliberately consumer-spending-focused. Findings here do not generalize to technology, healthcare, energy, or financial-sector equities, where the relevant seasonality drivers (earnings cycles, drug-approval calendars, geopolitical / climate events, central-bank meetings) are categorically different.

7.7 Survivorship handling is partial

We retain delisted tickers but do not attempt to recover their historical data when yfinance no longer serves it (TIF, GPS, SEAS, SIX, PARA, JWN under their pre-disposition tickers). A more complete survivorship treatment would source historical data for these tickers from an archival provider and include them as fully-populated rows. This is feasible and on the roadmap.

8. Future Work

The system as presented is a complete, working framework. Several extensions are natural and have been scoped:

1. **Confidence intervals via bootstrap resampling.** ~1000 resamples per (stock, month) cell would produce empirical CIs for each `avgReturn`, replacing the current implicit "trust the mean" with explicit uncertainty bands. Not machine learning; statistical inference within the existing descriptive framework.
2. **Market-relative returns.** Adding a market benchmark (SPY) to `PriceHistory`, computing returns over the same windows, and subtracting from the stock's return would isolate the seasonal-effect component from broader market drift.
3. **Backtest engine.** A configurable entry/exit-rule simulator with realistic transaction-cost assumptions, producing IRR and drawdown statistics on the historical data.
4. **Month-by-year aggregation.** The current heatmap shows month × window. The brief's longer-term goal was a month × year heatmap on each category page, requiring a new cached aggregation table.
5. **ADR FX adjustment.** A small FX-history table plus the relevant adjustments to ADR-ticker returns.
6. **Polygon.io as an alternative data source.** Replace yfinance for tickers it cannot serve, while preserving the `dataSource` column for auditability.
7. **Demo / paper-trading mode.** A non-real-money "trial position" feature that lets users open hypothetical positions on a stock from a chosen entry date and track unrealized P&L through subsequent events (see `notes/future-ideas.md` in the repository for the design sketch).

We deliberately do *not* plan to add machine-learning forecasting layers. As argued in Section 4.4, the sample sizes do not support it and the use case does not benefit from it; doing so would risk obscuring rather than illuminating the underlying patterns.

---

9. Conclusion

We have presented the design, methodology, and implementation of Stock Seasonality Analyzer: a reproducible, open-source web application for descriptive analysis of calendar and event-window seasonality in consumer-spending equities. The contribution is not a new statistical method or a new empirical finding, but a methodologically careful synthesis: survivorship-aware data ingestion, deterministic descriptive statistics with sample-size always surfaced, deliberate avoidance of machine learning given small samples, and aggressive caching of computed scores for sub-second user-facing pages.

Compared to commercial seasonality tools, the present system is fully transparent and reproducible: any user can clone the repository, point at a Postgres instance, and recompute every reported percentage from the underlying daily price data. We believe this transparency is valuable for educational and research contexts in which methodology is itself the subject of scrutiny.

Acknowledgments

This project was developed end-to-end over six phased iterations during May 2026, with each phase reviewed and approved before proceeding. Engineering and code generation were assisted by Anthropic's Claude AI under continuous human direction; methodological and design decisions are the author's responsibility. The author thanks the open-source maintainers of `next.js`, `prisma`, `yfinance`, `psycopg`, and `vitest` for the foundation libraries that made the system buildable in this timeframe.

References

Andrade, S. C., Chhaochharia, V., & Fuerst, M. E. (2013). "Sell in May and go away" just won't go away. *Financial Analysts Journal*, 69(4), 94–105.

Ariel, R. A. (1987). A monthly effect in stock returns. *Journal of Financial Economics*, 18(1), 161–174.

Bollerslev, T., Patton, A. J., & Quaedvlieg, R. (2009). Realized volatility forecasting: An empirical investigation. *Journal of Econometrics*.

Bouman, S., & Jacobsen, B. (2002). The Halloween indicator, "Sell in May and go away": Another puzzle. *The American Economic Review*, 92(5), 1618–1635.

Brown, S. J., Goetzmann, W., Ibbotson, R. G., & Ross, S. A. (1992). Survivorship bias in performance studies. *Review of Financial Studies*, 5(4), 553–580.

Brown, S. J., & Warner, J. B. (1985). Using daily stock returns: The case of event studies. *Journal of Financial Economics*, 14(1), 3–31.

Cao, M., & Wei, J. (2005). Stock market returns: A note on temperature anomaly. *Journal of Banking and Finance*, 29(6), 1559–1573.

Elton, E. J., Gruber, M. J., & Blake, C. R. (1996). Survivorship bias and mutual fund performance. *Review of Financial Studies*, 9(4), 1097–1120.

Haugen, R. A., & Lakonishok, J. (1988). *The Incredible January Effect: The Stock Market's Unsolved Mystery*. Dow Jones-Irwin.

Heston, S. L., & Sadka, R. (2008). Seasonality in the cross-section of stock returns. *Journal of Financial Economics*, 87(2), 418–445.

Keim, D. B. (1983). Size-related anomalies and stock return seasonality: Further empirical evidence. *Journal of Financial Economics*, 12(1), 13–32.

Kim, C. W., & Park, J. (1994). Holiday effects and stock returns: Further evidence. *Journal of Financial and Quantitative Analysis*, 29(1), 145–157.

Lakonishok, J., & Smidt, S. (1988). Are seasonal anomalies real? A ninety-year perspective. *Review of Financial Studies*, 1(4), 403–425.

MacKinlay, A. C. (1997). Event studies in economics and finance. *Journal of Economic Literature*, 35(1), 13–39.

Rozeff, M. S., & Kinney, W. R. (1976). Capital market seasonality: The case of stock returns. *Journal of Financial Economics*, 3(4), 379–402.

Appendix A. Reproducibility

The complete source code, schema, seed data, ingestion scripts, and analysis library are available in the project repository. To reproduce the results in this paper from scratch:

```
# 1. Clone and install dependencies
git clone <repository-url> stock-seasonality
cd stock-seasonality
npm install

# 2. Configure a PostgreSQL connection
cp .env.example .env
# Edit .env to point DATABASE_URL at your Postgres instance

# 3. Apply schema migrations
npx prisma migrate dev

# 4. Seed the universe (categories, stocks, festivals)
npm run db:seed

# 5. Set up the Python ingestion environment
python -m venv .venv
source .venv/Scripts/activate  # or .venv/bin/activate on macOS/Linux
pip install -r requirements.txt

# 6. Backfill 15 years of daily prices (takes ~90 seconds)
python ingest_prices.py --window 15y --include-delisted

# 7. Compute and cache all seasonality and event-window scores
npm run analyze

# 8. Run the unit test suite
npm test

# 9. Start the web application
npm run dev
# Then visit http://localhost:3000
```

All cached score values for any (stock, month, window, COVID-flag) tuple are deterministic given the same input price data; re-running the analysis pipeline against the same `PriceHistory` rows produces byte-identical results.

Word count: approximately 4,800 words including appendices. Tables and code blocks excluded from the prose count.